

Класс Pipeline

Паточенко Евгений

НИУ ВШЭ

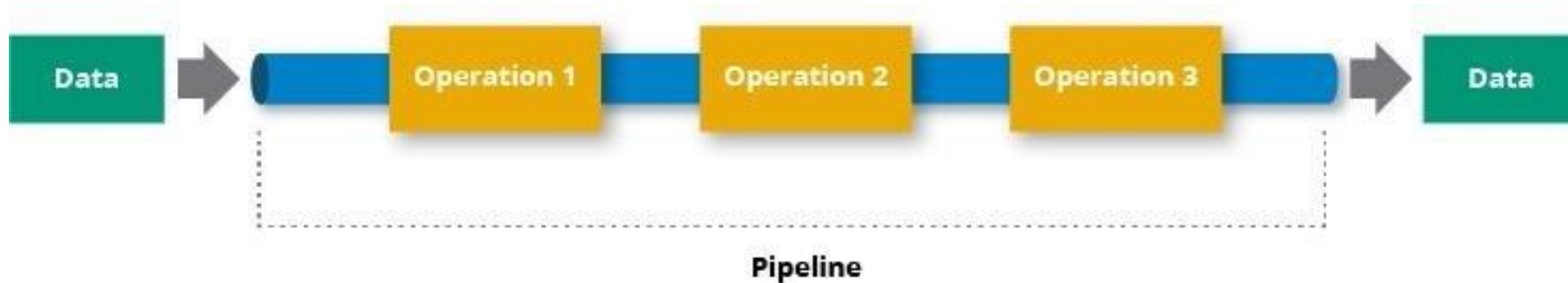
План занятия

- Возможности Pipeline
- Особенности применения
- Пример до/после
- Ключевые выводы

Pipeline

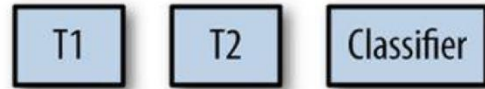
Возможности

- Объединить несколько действий в один объект
- Убрать дублирование кода
- Работать с пайплайном как объектом sklearn, вызывая стандартные функции .fit и .predict
- Сохранить пайплайн в виде целого объекта для дальнейшего переиспользования

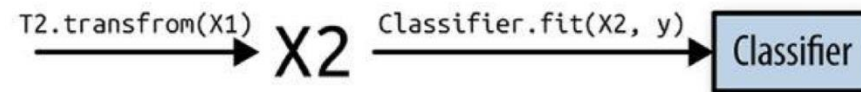
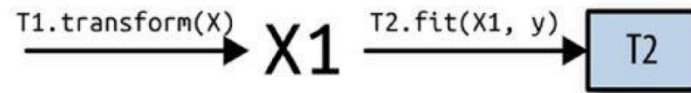
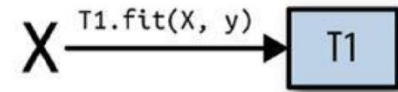


Pipeline

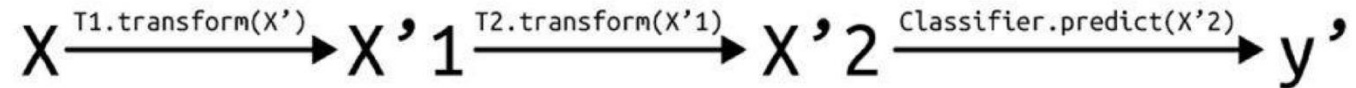
```
pipe = make_pipeline(T1(), T2(), Classifier())
```



```
pipe.fit(X, y)
```



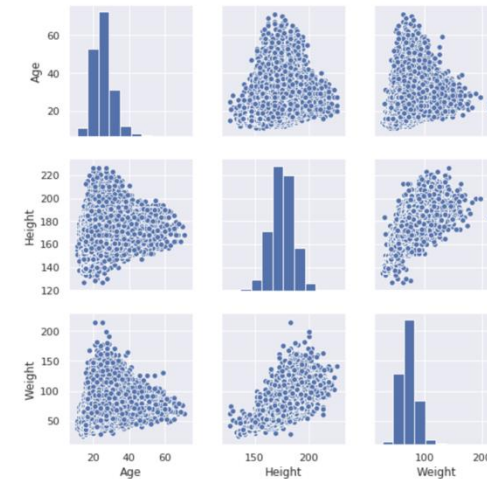
```
pipe.predict(X')
```



Pipeline

Применение

Да	Нет
EDA завершен и модель готова для фиксации и тюнинга	Стадия EDA
Модель готова для передачи в продакшн	Нет понимания итоговой структуры модели



Пример

Что у нас есть

Будем предсказывать уровень счастья. Это задача регрессии — предсказания численного значения по совокупности признаков

Признаки могут быть разных типов

В признаках есть пропуски и выбросы

Что мы хотим

Модель должна быть готова для переобучения на вновь поступивших данных

Модель должна быть готова к кросс-валидации

Модель должна быть сохранена как единый объект

Пример без использования Pipeline

Загрузим данные и посмотрим на признаки

```
file_path = 'https://github.com/evgpat/datasets/raw/main/responses.csv'
```

```
data = pd.read_csv(file_path)
```

```
interesting_features = ["Happiness in life", "Age", "Height", "Gender", "Weight", "Left -  
right handed", "Village - town", "Getting up", "God", "Health", "Borrowed stuff",  
"Self-criticism", "Elections", "Smoking", "Alcohol",  
"Number of friends", "Spending on healthy eating", "Music", "Movies", "Friends  
versus money", "Changing the past"]
```

```
target = ["Happiness in life"]
```

Загружаем данные

Отбираем признаки
для моделирования

Пример без использования Pipeline

Посмотрим какие есть признаки

численные признаки

```
numerical_features = ['Age', 'Height', 'Weight']
```

бинарные признаки

```
to_dummies = ['Gender', 'Left - right handed', 'Village - town', 'Smoking', "Alcohol"]
```

категориальные признаки

```
categorical_features = [col for col in data.columns if col not in numerical_features + target + to_dummies]
```

Пример без использования Pipeline

Преобразуем признаки для модели

```
# нормировка для численных признаков
scaler = StandardScaler()
data_train_scaled = scaler.fit_transform(data_train[numerical_features])

# кодирование для категориальных признаков
ohe = OneHotEncoder(sparse=False, handle_unknown='ignore')
data_train_ohe = ohe.fit_transform(data_train[to_dummies])

# объединение признаков после предобработки
data_train_transformed = pd.concat(
    [pd.DataFrame(data_train_scaled, columns=numerical_features),
     pd.DataFrame(data_train_ohe, columns=ohe.get_feature_names())],
    axis=1)
```

Пример без использования Pipeline

Создадим модель

```
model = LogisticRegression()  
model.fit(data_train_transformed, data_train[target])  
  
preds = model.predict(data_train_transformed)
```

	precision	recall	f1-score	support
-1	0.52	0.07	0.12	259
0	0.51	0.97	0.67	374
1	0.00	0.00	0.00	107
accuracy			0.51	740
macro avg	0.34	0.34	0.26	740
weighted avg	0.44	0.51	0.38	740

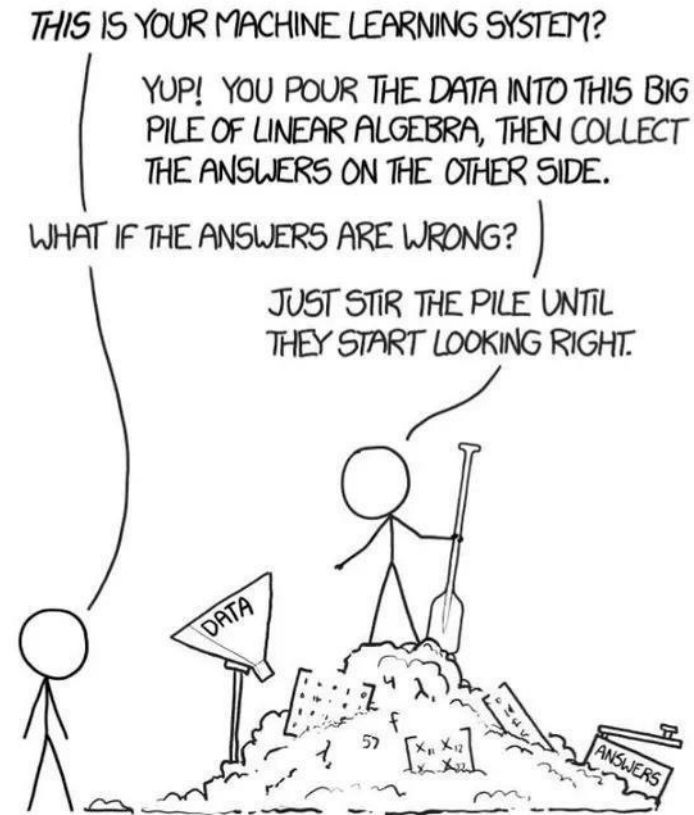
Пример без использования Pipeline

Для валидации необходимо будет проделать
все эти шаги **еще раз** на тестовой выборке

Пример без использования Pipeline

Итого:

- Дублируется код
- Сложно масштабировать
- Сложно поддерживать код
- Работа занимает много времени



Пример с Pipeline

Предобработка признаков

для числовых - применяем SimpleImputer, а затем StandardScaler

```
numerical_transformer = Pipeline(steps=[  
    ("imputer", SimpleImputer()),  
    ("scaler", StandardScaler()),  
    ("fs", SelectKBest(score_func=f_classif, k="all")),  
])
```

для категориальных - применяем SimpleImputer, а затем OneHotEncoder

```
categorical_transformer = Pipeline(steps=[  
    ("imputer", SimpleImputer()),  
    ("onehot", OneHotEncoder(handle_unknown="ignore")),  
])
```

собираем воедино трансформеры для числовых и категориальных признаков

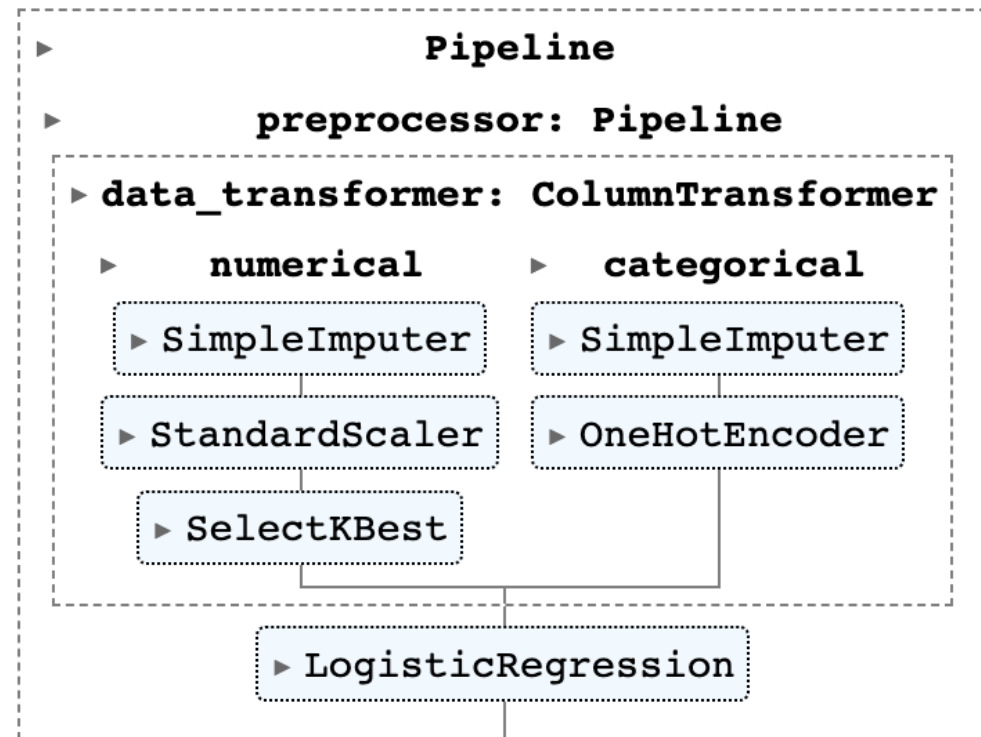
```
data_transformer = ColumnTransformer(transformers=[  
    ("numerical", numerical_transformer, numerical_features),  
    ("categorical", categorical_transformer, categorical_features),  
])
```

Пример с Pipeline

Соберем пайплайн

```
# создание конвейера препроцессора, который сначала
# преобразует данные
preprocessor = Pipeline(
    steps=[
        ("data_transformer", data_transformer)
    ])

classifier_pipeline = Pipeline(
    steps=[
        ("preprocessor", preprocessor),
        ("classifier", LogisticRegression())
    ])
```



Пример с Pipeline (еще раз по шагам)

для числовых - применяем SimpleImputer, а затем StandardScaler

```
numerical_transformer = Pipeline(steps=[  
    ("imputer", SimpleImputer()),  
    ("scaler", StandardScaler()),  
    ("fs", SelectKBest(score_func=f_classif, k="all"))])
```

для категориальных - применяем SimpleImputer, а затем OneHotEncoder

```
categorical_transformer = Pipeline(steps=[  
    ("imputer", SimpleImputer()),  
    ("onehot", OneHotEncoder(handle_unknown="ignore"))])
```

собираем воедино трансформеры для числовых и категориальных признаков

```
data_transformer = ColumnTransformer(transformers=[  
    ("numerical", numerical_transformer, numerical_features),  
    ("categorical", categorical_transformer, categorical_features)])
```

создание конвейера препроцессора, который сначала преобразует данные

```
preprocessor = Pipeline(  
    steps=[  
        ("data_transformer", data_transformer)])
```

```
classifier_pipeline = Pipeline(  
    steps=[  
        ("preprocessor", preprocessor),  
        ("classifier", LogisticRegression())])
```

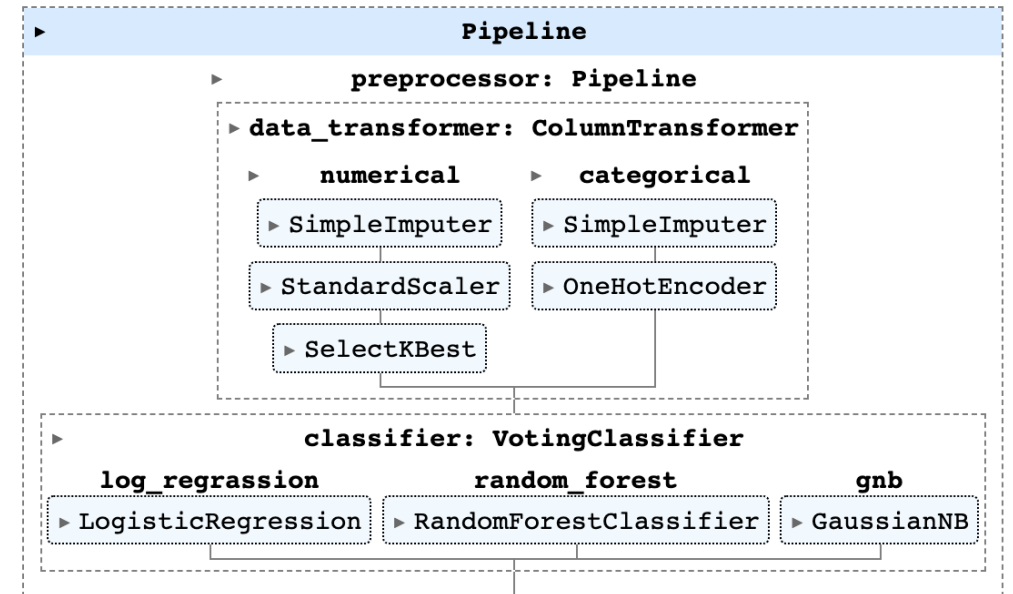
Пример с Pipeline

Добавляем модели

```
clf1 = LogisticRegression(multi_class="multinomial", random_state=1)
clf2 = RandomForestClassifier(n_estimators=50, random_state=1)
clf3 = GaussianNB()
```

```
blending_classifier = VotingClassifier(
    estimators=[
        ("log_regression", clf1),
        ("random_forest", clf2),
        ("gnb", clf3)])
```

```
classifier_pipeline = Pipeline(
    steps=[
        ("preprocessor", preprocessor),
        ("classifier", blending_classifier)])
```



Пример с Pipeline

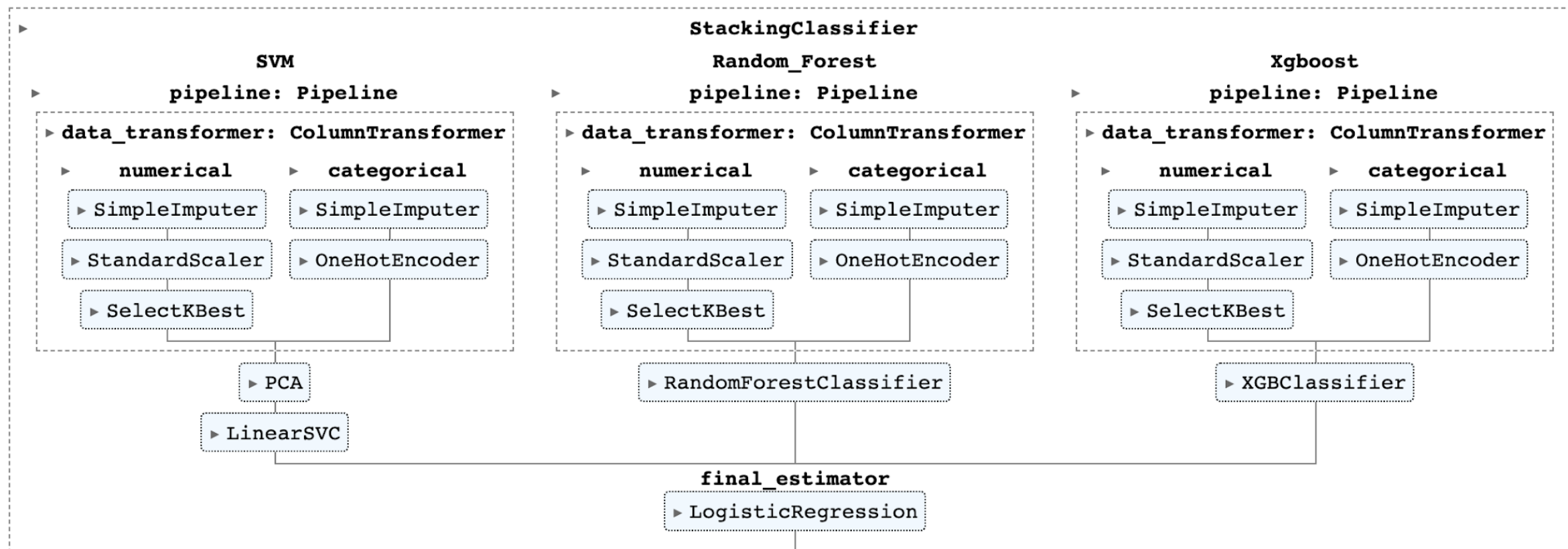
Добавляем стекинг

```
# получим список эстиматоров
estimators = [
    ("SVM", make_pipeline(preprocessor, PCA(), LinearSVC())),
    ("Random_Forest", make_pipeline(preprocessor, RandomForestClassifier())),
    ("Xgboost", make_pipeline(preprocessor, xgboost.XGBClassifier())),
]

# построим классификатор, используя стекинг и наш список классификаторов
stacking_classifier = StackingClassifier(
    estimators=estimators,
    final_estimator=LogisticRegression(n_jobs=-1, verbose=True),
    n_jobs=-1,
    verbose=True,
)
```

Пример с Pipeline

Финальный классификатор



Пример с Pipeline

Сохранение и загрузка сложной модели

```
# Для сериализации моделей
import pickle

# Сохранение модели
with open("awesome_pipeline.pkl", "wb") as f:
    pickle.dump(stacking_classifier, f)

# Загрузка модели
with open("awesome_pipeline.pkl", 'rb') as f:
    pipeline_from_saved = pickle.load(f)
```



Ключевые выводы

- Пайплайн помогает строить сложные модели быстрее и сохранять единообразие
- Пайплайн следует применять на стадии перехода от EDA к обучению (дообучению)
- Пайплайн помогает стабилизировать модель и подготовить ее к переводу в продакшн

Полезные ссылки

- [Раздел `sklearn.pipeline`](#) — обзор модуля, примеры построения и комбинирования шагов обработки данных
- [Документация класса `Pipeline` в `scikit-learn`](#) — описание параметров, методов, примеры использования и особенности работы конвейера